CSC 3320: System-Level Programming

Chapter 11: Pointers-II and Chapter 12

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Review of Exercises

- If `i` is a variable and p points to `i`, which of the following expressions are aliases for `i`?

  A) *p B) *&p C) *i D) *&i

  E) &p F) &*p G) &i H) &*i

  AD

- If `i` is an `int` variable and `p` and `q` are pointers to `int`, which of the following assignments are legal?

  CDE

  A) p=i; B) p=&q; C) *p=*q D) p=q

  E) p=*&q; F) &p=q; G) *p=&i H) *p=q

Exercises

- If `i` is a variable and p points to `i`, which of the following expressions are aliases for `i`?

AG

a) *p - Correct
b) &p - address of p
c) *&p - address of i
d) &*p - address of i
e) *i - Error 'unary *'
f) &i - address of i
g) *&i - Corect
h) &*i - Error 'unary *'

Exercises

- If `i` is an `int` variable and `p` and `q` are pointers to `int`, which of the following assignments are legal?

a) **p = i;** warning: assignment makes pointer from integer without a cast
b) ***p = &i;** warning: assignment makes integer from pointer without a cast
c) **&p = q;** Error invalid lvalue (Can't change the address of p)
d) **p = &q;** warning: assignment from incompatible pointer type (Not int)
e) **p = *&q;** Perfect - p gets the value of q.
f) **p = q;** Perfect.
g) **p = *q;** p is assigned what q points to.
h) ***p = q;** warning: assignment makes integer from pointer without a cast
i) ***p = *q;**

Using const to Protect Arguments

- When we call a function and pass it a pointer to a variable, we normally assume that the function will modify the variable.
- Sometimes we just want to examine the value of a variable, not change it.
- We can use the word `const` to document that a function won't change an object whose pointer is passed to the function.

```
void f(const int *p)
{
  *p = 0; // wrong: p is a pointer to a "constant integer"
}
```

Pointers as Return Values

- Return the location of an answer instead of returning its value.

```
int *max(int *a, int *b)
{
    if(*a > *b)
        return a;
    else
        return b;
}

int *p, x, y;
P = max(&x, &y);
```

If x > y, return the address of x otherwise, return the address of y

Pointers as Return Values

- Be careful! Please do not return a pointer to a local variable.

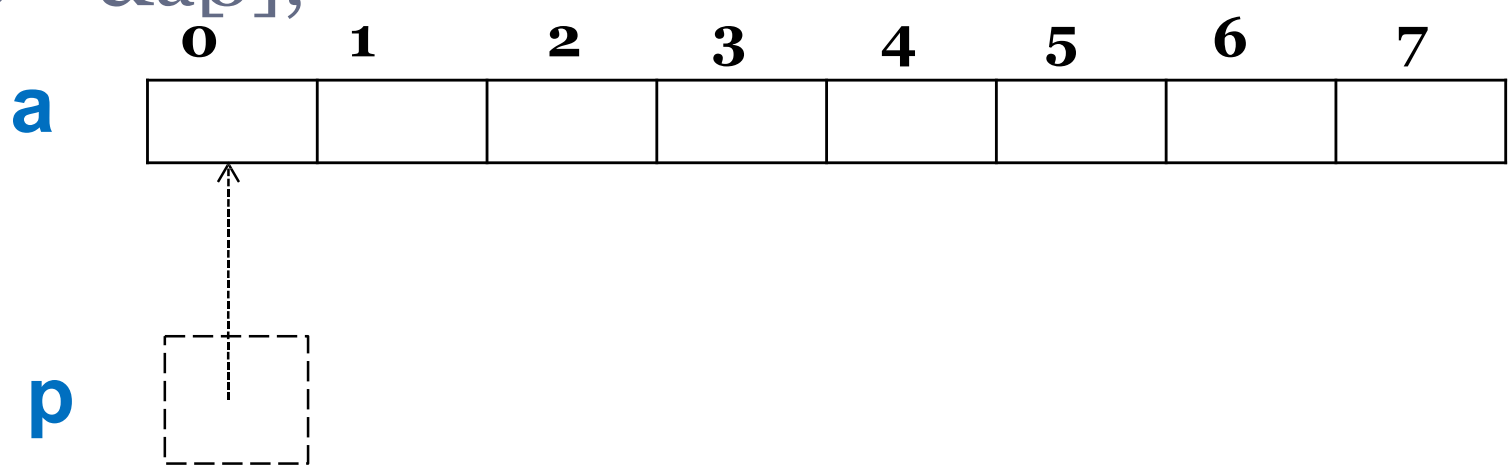
```
int *f(void)
{
        int a;
        ……
        return &a;
}
```

Once f returns, the variable a does not exist, so the pointer to it will be invalid.

Chapter 12: Pointers and Arrays

Pointer Arithmetic

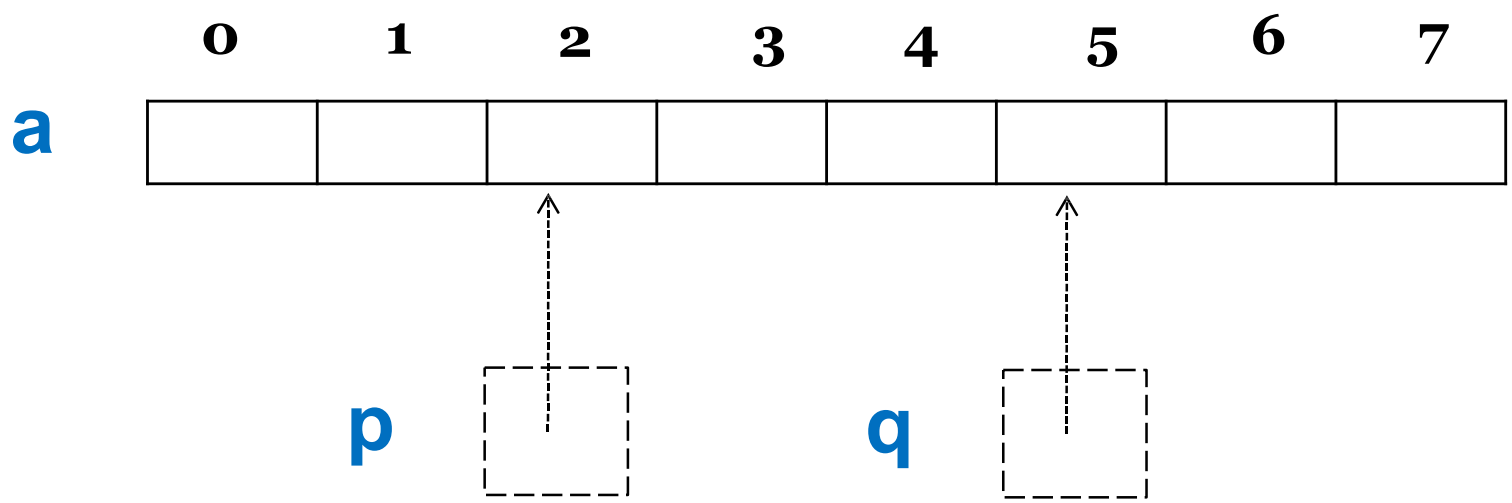
- Pointers can point to array elements, not just ordinary variables.
- Example
  - int a[8], *p;
  - p = &a[0];



Pointer Arithmetic

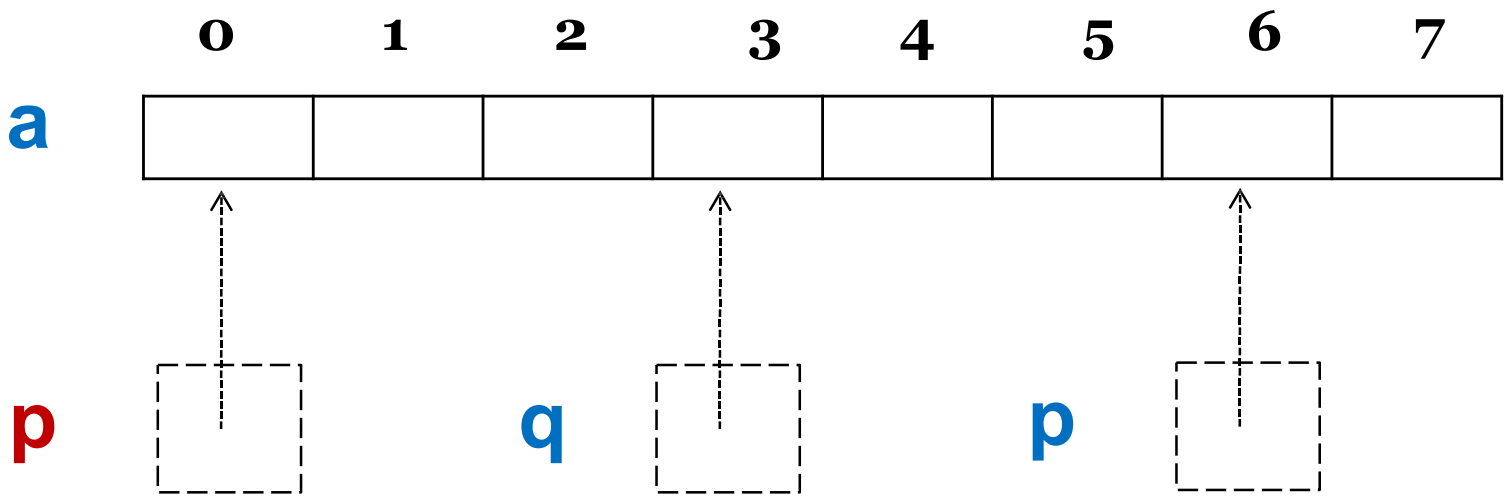
- Add an integer to a pointer
  - p = &a[2];
  - q = p + 3;
  - p += 4;

When index is out of bound, accessing to *p may generate unexpected result



Pointer Arithmetic

- Subtract an integer from a pointer
  - p = &a[6];
  - q = p - 3;
  - p -= 6;



Use Pointers for Array Processing

- Visit the elements of an array by repeatedly incrementing a pointer variable.
- Example: sum the elements of an array a

```
#define N 10

int a[N], sum, *p;
sum = 0;
for(p = &a[0]; p < &a[N]; p++)
    sum += *p;
```

Combing the * and ++ Operators

- *p++ or *(p++)
- (*p)++
- *++p or *(++p)
- ++*p or ++(*p)
- Note: The postfix version of ++ takes precedence over *

See the next slide for a more detailed example

Example

```
int main()
{
   int a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
   int *p, q, r, s, t, u;

   p = &a[3];
   q = *p++; // assigns 4. same as *(p++). Uses the value first and then increment p
   p = &a[3];
   r = *(p++); // assigns 4 first then increments the pointer, same as before.
   p = &a[3];
   s = (*p)++; // assigns 4 first, increments the value next. So, a[3] becomes 5
   p = &a[3];
   t = *++p;  // increments p first and then access the element. So, assigns 5
   p = &a[3];
   u = ++*p;  // accesses the current first (5), increases the value, and then assigns 6.

   printf("Current: %d   Next element: %d\n", *p, *(p+1));  // 6 5

   printf("  *ptr++= %4d\n", q);  // 4
   printf("*(ptr++)= %4d\n", r);  // 4
   printf("(*ptr)++= %4d\n", s);  // 4
   printf("  *++ptr= %4d\n", t);  // 5
   printf("  ++*ptr= %4d\n", u);  // 6

   return 0;
}
```

Using an Array Name as a Pointer

- The name of an array can be used as a pointer to the first element in the array.
- Example

```
int a[10];
*a = 7;    // modify a[0]
*(a + 2) = 13; // modify a[2]

while(*a  != 0 )
      a++;          // wrong
```

Using an Array Name as a Pointer

- The two functions below are equivalent

```
int find_largest(int a[], int n)
{
    . . .
}

int find_largest(int *a,  int n)
{
    . . .
}
```

Using an Array Name as a Pointer

- A new version of calculating sum of an array

```
#define N 10
…..
int a[N], i, sum = 0, *p = a;
…..
for(i=0; i< N;i++)
      sum += p[i];
```